

# BERT-Based Sentiment Analysis Using a Pre-Trained Model

## Deep Learning Practical

### Objective:

To implement a pre-trained BERT model for sentiment analysis using a sample dataset, fine-tune the model, evaluate its performance, and classify custom text reviews as positive or negative.

### Technologies Used

- Python
- Google Colab
- PyTorch
- Hugging Face Transformers
- Hugging Face Datasets
- Scikit-learn
- Matplotlib
- Seaborn

### Model

bert-base-uncased

### Dataset

IMDB Movie Review Dataset

### Classification

- 0 → Negative
- 1 → Positive

## 1. Introduction

BERT stands for **Bidirectional Encoder Representations from Transformers**. It is a Transformer-based pre-trained language model that learns contextual representations of words from large text corpora.

In this practical, a pre-trained BERT model is fine-tuned for **binary sentiment classification**. The model receives movie reviews from the IMDB dataset and predicts

whether each review is positive or negative.

## Overall Workflow

```
IMDB Dataset
  ↓
Data Loading
  ↓
Data Sampling
  ↓
BERT Tokenization
  ↓
Pre-trained BERT
  ↓
Fine-Tuning
  ↓
Evaluation
  ↓
Confusion Matrix and Classification Report
  ↓
Custom Text Prediction
```

## ✓ 2. Install Required Libraries

Run this cell first in Google Colab.

```
!pip install -q -U transformers datasets evaluate accelerate scikit-learn se
```

```
_____ 12.3/12.3 MB 44.7 MB/s eta 0:00:0
_____ 559.1/559.1 kB 26.6 MB/s eta 0:00
_____ 84.1/84.1 kB 3.7 MB/s eta 0:00:00
_____ 394.3/394.3 kB 19.2 MB/s eta 0:00
_____ 9.1/9.1 MB 45.7 MB/s eta 0:00:00
```

## ✓ 3. Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import torch

from datasets import load_dataset
```

```
from transformers import (  
    AutoTokenizer,  
    AutoModelForSequenceClassification,  
    TrainingArguments,  
    Trainer  
)  
  
from sklearn.metrics import (  
    confusion_matrix,  
    classification_report  
)  
  
import evaluate  
  
print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

## ✓ 4. Check GPU Availability

BERT fine-tuning is faster when a GPU is available. In Google Colab, select **Runtime** → **Change runtime type** → **T4 GPU**.

```
print("PyTorch version:", torch.__version__)  
print("GPU available:", torch.cuda.is_available())  
  
if torch.cuda.is_available():  
    print("GPU:", torch.cuda.get_device_name(0))  
else:  
    print("Running on CPU")
```

```
PyTorch version: 2.11.0+cu128  
GPU available: True  
GPU: Tesla T4
```

## ✓ 5. Load the IMDB Dataset

The IMDB dataset contains movie reviews with binary sentiment labels. The current Hugging Face repository identifier is `stanfordnlp/imdb`.

```
dataset = load_dataset("stanfordnlp/imdb")  
  
print(dataset)
```

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
README.md: 100% 7.81k/7.81k [00:00<00:00, 640kB/s]

plain_text/train-00000-of- 21.0MB / 21.0MB, 2.04MB/s
00001.parquet: reconstructing file: 100%

plain_text/train-00000-of- 20.9MB, 2.02MB/s
00001.parquet: downloading bytes:

plain_text/test-00000-of- 20.5MB / 20.5MB, 2.00MB/s
00001.parquet: reconstructing file: 100%

plain_text/test-00000-of- 20.4MB, 1.99MB/s
00001.parquet: downloading bytes:

plain_text/unsupervised-00000-of- 42.0MB / 42.0MB, 4.06MB/s
00001.p(...): reconstructing file: 100%

plain_text/unsupervised-00000-of- 41.9MB, 4.03MB/s
00001.p(...): downloading bytes:

Generating train split: 100% 25000/25000 [00:00<00:00, 64674.00 examples/s]
Generating test split: 100% 25000/25000 [00:00<00:00, 60748.44 examples/s]
Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 96154.37 examples/s]
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
})

```

## ✓ 6. Inspect a Sample Review

```

print("Sample Review:")
print(dataset["train"][0]["text"])

print("\nLabel:")
print(dataset["train"][0]["label"])

print("\nLabel Meaning:")
print("0 = Negative")
print("1 = Positive")

```

```

Sample Review:
I rented I AM CURIOUS-YELLOW from my video store because of all the controver

Label:
0

```

```
Label Meaning:  
0 = Negative  
1 = Positive
```

## ✓ 7. Create a Smaller Dataset

The original IMDB dataset contains many reviews. To make the practical faster, we use:

- **5,000 training reviews**
- **1,000 testing reviews**

A fixed random seed is used so that the selected samples are reproducible.

```
train_dataset = dataset["train"].shuffle(seed=42).select(range(5000))  
test_dataset = dataset["test"].shuffle(seed=42).select(range(1000))  
  
print("Training samples:", len(train_dataset))  
print("Testing samples:", len(test_dataset))
```

```
Training samples: 5000  
Testing samples: 1000
```

## ✓ 8. Load the Pre-Trained BERT Tokenizer

The tokenizer converts text into numerical tokens that can be processed by BERT.

We use:

```
bert-base-uncased
```

The model is uncased, meaning that it does not distinguish between uppercase and lowercase letters.

```
model_name = "bert-base-uncased"  
  
tokenizer = AutoTokenizer.from_pretrained(model_name)  
  
print("Tokenizer loaded successfully.")
```

```

config.json: 100% 570/570 [00:00<00:00, 28.2kB/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 3.74kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 4.99MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 6.94MB/s]
Tokenizer loaded successfully.

```

## ✓ 9. Demonstrate BERT Tokenization

```

text = "I loved this movie."

tokens = tokenizer.tokenize(text)

print("Original text:")
print(text)

print("\nTokens:")
print(tokens)

print("\nToken IDs:")
print(tokenizer.convert_tokens_to_ids(tokens))

```

```

Original text:
I loved this movie.

Tokens:
['i', 'loved', 'this', 'movie', '.']

Token IDs:
[1045, 3866, 2023, 3185, 1012]

```

## ✓ 10. Tokenize the Dataset

Each review is converted into:

- `input_ids` — numerical token IDs
- `attention_mask` — identifies actual tokens and padding
- `token_type_ids` — segment information used by BERT when applicable

Reviews are truncated or padded to a maximum length of 128 tokens.

```

def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",

```

```
        truncation=True,
        max_length=128
    )

    train_tokenized = train_dataset.map(
        tokenize_function,
        batched=True
    )

    test_tokenized = test_dataset.map(
        tokenize_function,
        batched=True
    )

    print("Tokenization completed.")
```

```
Map: 100%                               5000/5000 [00:07<00:00, 656.89 examples/s]
Map: 100%                               1000/1000 [00:01<00:00, 853.66 examples/s]
Tokenization completed.
```

## ✓ 11. Inspect Tokenized Data

```
print(train_tokenized[0])
```

```
{'text': 'There is no relation at all between Fortier and Profiler but the fa
```

## ✓ 12. Load the Pre-Trained BERT Classification Model

`AutoModelForSequenceClassification` adds a classification head on top of BERT.

Since this is binary sentiment classification:

- `0` → Negative
- `1` → Positive
- `num_labels = 2`

```
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)

print("BERT classification model loaded successfully.")
```

```

model.safetensors: reconstructing file: 100% 440MB / 440MB, 37.1MB/s

model.safetensors: downloading bytes: 415MB, 36.7MB/s

Loading weights: 100% 199/199 [00:00<00:00, 4879.24it/s]
[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-unc
Key | Status |
-----+-----+
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
classifier.weight | MISSING |
classifier.bias | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture;
- MISSING: those params were newly initialized because missing from the chec
BERT classification model loaded successfully.

```

### ✓ 13. Move the Model to GPU or CPU

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model.to(device)

print("Using device:", device)

```

Using device: cuda

### ✓ 14. Define Evaluation Metric

Accuracy is used as the main evaluation metric.

```
accuracy = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)

    return accuracy.compute(
        predictions=predictions,
        references=labels
    )
```

Downloading builder script: 4.20k/? [00:00<00:00, 255kB/s]

## ✓ 15. Define Training Arguments

Important parameters:

- Learning rate: `2e-5`
- Batch size: `8`
- Epochs: `2`
- Weight decay: `0.01`

These settings provide a practical balance between training time and model performance.

```
training_args = TrainingArguments(
    output_dir="./bert_sentiment_results",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_strategy="steps",
    logging_steps=100,
    load_best_model_at_end=True,
    report_to="none"
)

print("Training arguments configured.")
```

Training arguments configured.

## ✓ 16. Create the Trainer

```

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tokenized,
    eval_dataset=test_tokenized,
    processing_class=tokenizer,
    compute_metrics=compute_metrics
)

print("Trainer created successfully.")

```

Trainer created successfully.

## ✓ 17. Fine-Tune the BERT Model

This is the main training step. The pre-trained BERT model is fine-tuned using the labeled IMDB reviews.

```
train_result = trainer.train()
```

 [1250/1250 04:39, Epoch 2/2]

| Epoch | Training Loss | Validation Loss | Accuracy |
|-------|---------------|-----------------|----------|
| 1     | 0.370780      | 0.375786        | 0.857000 |
| 2     | 0.234163      | 0.537751        | 0.851000 |

Writing model shards: 100% 1/1 [00:05<00:00, 5.68s/it]

Writing model shards: 100% 1/1 [00:11<00:00, 11.20s/it]

[transformers] There were missing keys in the checkpoint model loaded: ['bert  
[transformers] There were unexpected keys in the checkpoint model loaded: ['b

## ✓ 18. Evaluate the Fine-Tuned Model

```

results = trainer.evaluate()

print("Evaluation Results:")
for key, value in results.items():
    print(f"{key}: {value}")

```

[125/125 00:07]

| Training Loss | Validation Loss | Epoch | Accuracy |
|---------------|-----------------|-------|----------|
| 0.234163      | 0.376670        | 2     | 0.857000 |

Evaluation Results:  
eval\_loss: 0.3766695559024811  
eval\_accuracy: 0.857

## ✓ 19. Display Training History

```
history = trainer.state.log_history
history_df = pd.DataFrame(history)

display(history_df)
```

|    | loss     | grad_norm | learning_rate | epoch | step | eval_loss | eval_accuracy |
|----|----------|-----------|---------------|-------|------|-----------|---------------|
| 0  | 0.585708 | 25.559496 | 1.841600e-05  | 0.16  | 100  | NaN       | NaN           |
| 1  | 0.399240 | 36.678967 | 1.681600e-05  | 0.32  | 200  | NaN       | NaN           |
| 2  | 0.374515 | 5.216481  | 1.521600e-05  | 0.48  | 300  | NaN       | NaN           |
| 3  | 0.413757 | 11.722837 | 1.361600e-05  | 0.64  | 400  | NaN       | NaN           |
| 4  | 0.386491 | 4.292809  | 1.201600e-05  | 0.80  | 500  | NaN       | NaN           |
| 5  | 0.370780 | 17.003296 | 1.041600e-05  | 0.96  | 600  | NaN       | NaN           |
| 6  | NaN      | NaN       | NaN           | 1.00  | 625  | 0.375786  | 0.857         |
| 7  | 0.247274 | 2.351737  | 8.816000e-06  | 1.12  | 700  | NaN       | NaN           |
| 8  | 0.221533 | 0.697629  | 7.216000e-06  | 1.28  | 800  | NaN       | NaN           |
| 9  | 0.268757 | 5.613137  | 5.616000e-06  | 1.44  | 900  | NaN       | NaN           |
| 10 | 0.241249 | 37.123123 | 4.016000e-06  | 1.60  | 1000 | NaN       | NaN           |
| 11 | 0.238974 | 3.708276  | 2.416000e-06  | 1.76  | 1100 | NaN       | NaN           |
| 12 | 0.234163 | 0.193694  | 8.160000e-07  | 1.92  | 1200 | NaN       | NaN           |
| 13 | NaN      | NaN       | NaN           | 2.00  | 1250 | 0.537751  | 0.851         |
| 14 | NaN      | NaN       | NaN           | 2.00  | 1250 | NaN       | NaN           |
| 15 | NaN      | NaN       | NaN           | 2.00  | 1250 | 0.376670  | 0.857         |

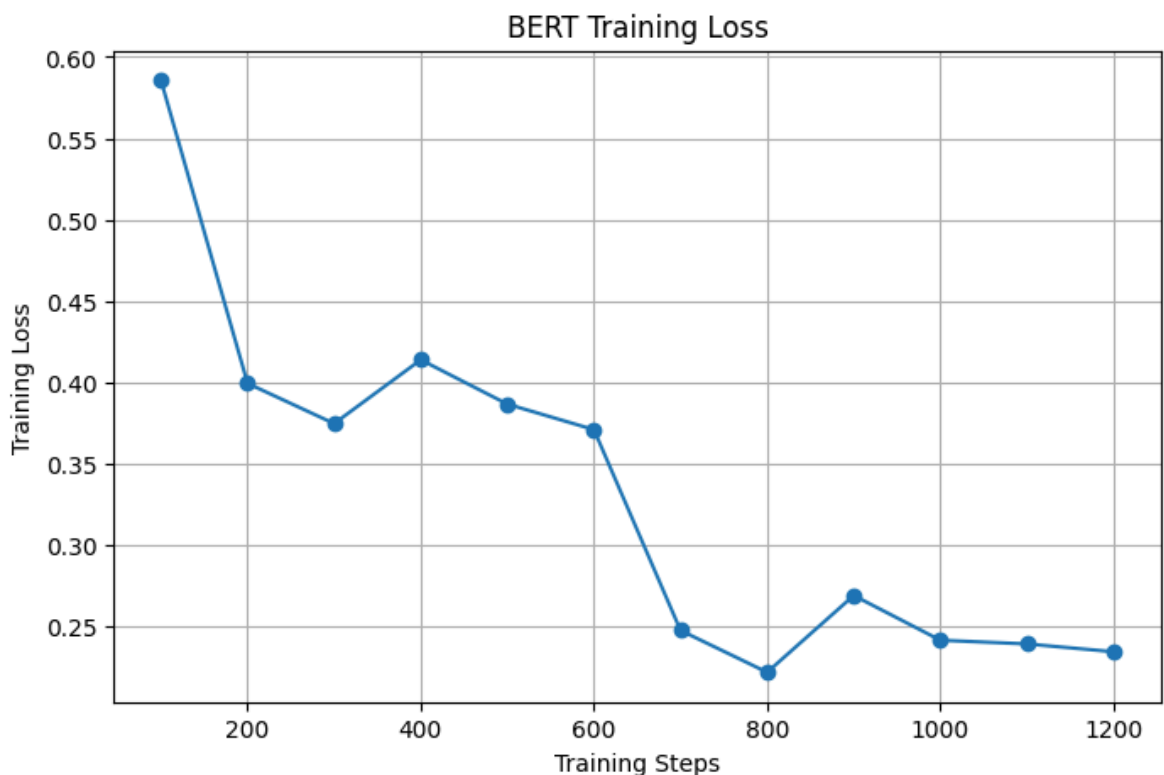
Next steps:

[Generate code with history\\_df](#)[New interactive sheet](#)

## 20. Plot Training Loss

```
train_loss = history_df[history_df["loss"].notna()]

plt.figure(figsize=(8, 5))
plt.plot(train_loss["step"], train_loss["loss"], marker="o")
plt.xlabel("Training Steps")
plt.ylabel("Training Loss")
plt.title("BERT Training Loss")
plt.grid(True)
plt.show()
```

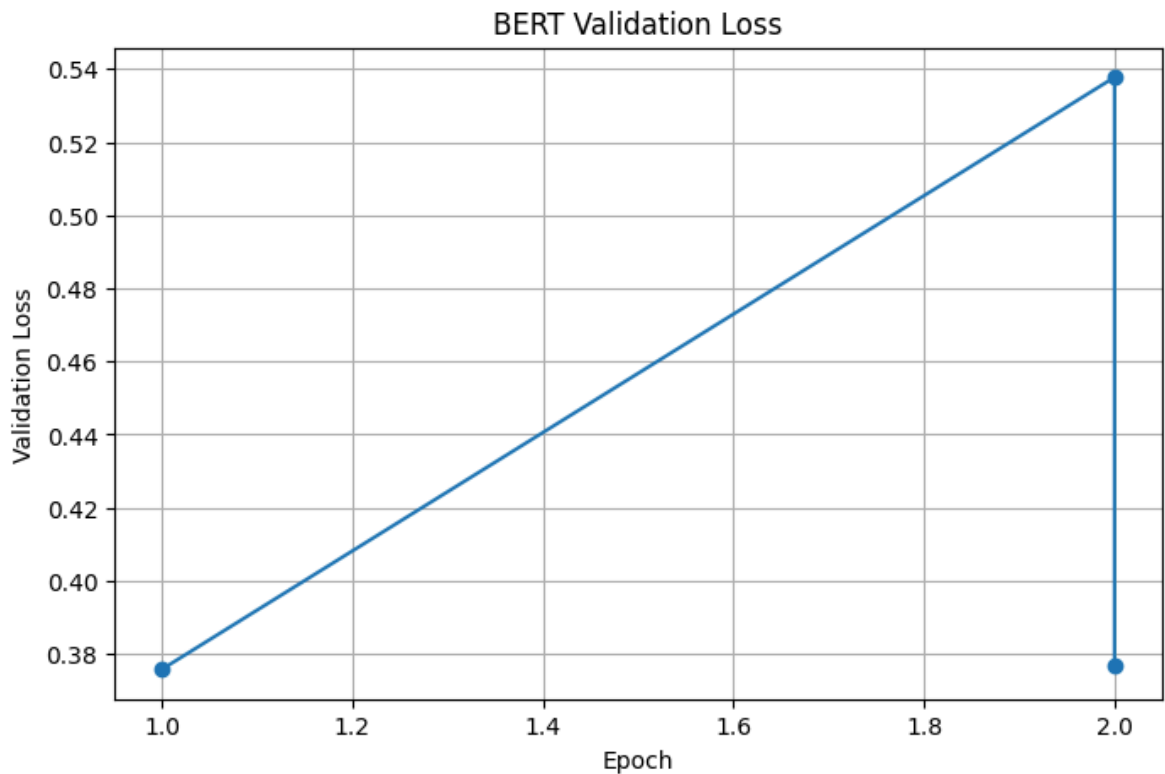


## 21. Plot Validation Loss

```
eval_loss = history_df[history_df["eval_loss"].notna()]

plt.figure(figsize=(8, 5))
plt.plot(eval_loss["epoch"], eval_loss["eval_loss"], marker="o")
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.title("BERT Validation Loss")
plt.grid(True)
```

```
plt.show()
```



## ✓ 22. Generate Predictions on the Test Dataset

The fine-tuned model is used to predict sentiment labels for the complete selected test set.

```
prediction_output = trainer.predict(test_tokenized)

predicted_labels = np.argmax(
    prediction_output.predictions,
    axis=1
)

actual_labels = prediction_output.label_ids

print("Predictions generated successfully.")
```

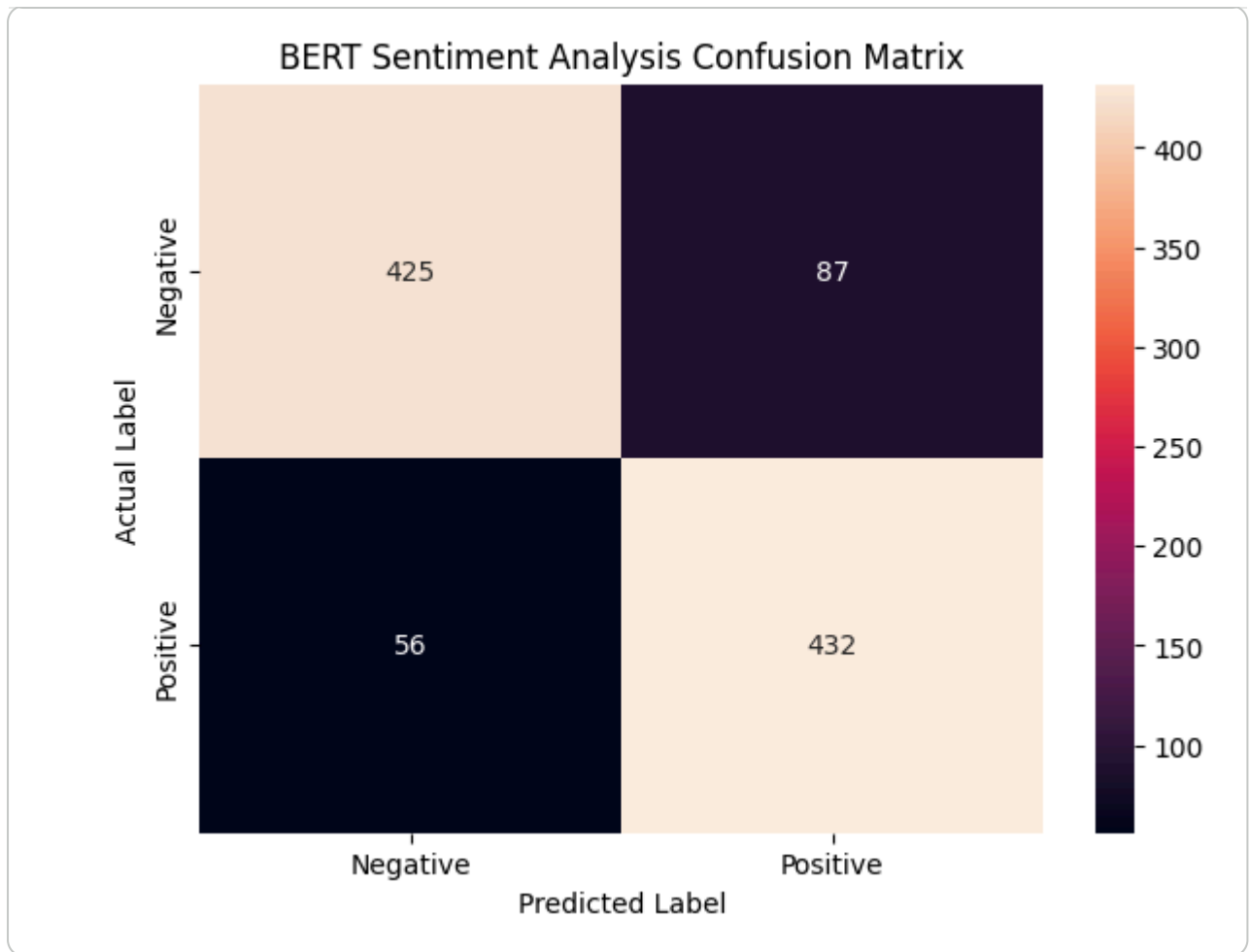
Predictions generated successfully.

## ✓ 23. Confusion Matrix

```
cm = confusion_matrix(  
    actual_labels,  
    predicted_labels  
)  
  
print("Confusion Matrix:")  
print(cm)
```

```
Confusion Matrix:  
[[425  87]  
 [ 56 432]]
```

```
plt.figure(figsize=(7, 5))  
  
sns.heatmap(  
    cm,  
    annot=True,  
    fmt="d",  
    xticklabels=["Negative", "Positive"],  
    yticklabels=["Negative", "Positive"]  
)  
  
plt.xlabel("Predicted Label")  
plt.ylabel("Actual Label")  
plt.title("BERT Sentiment Analysis Confusion Matrix")  
  
plt.show()
```



## ✓ 24. Classification Report

```
report = classification_report(  
    actual_labels,  
    predicted_labels,  
    target_names=["Negative", "Positive"]  
)  
  
print(report)
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Negative     | 0.88      | 0.83   | 0.86     | 512     |
| Positive     | 0.83      | 0.89   | 0.86     | 488     |
| accuracy     |           |        | 0.86     | 1000    |
| macro avg    | 0.86      | 0.86   | 0.86     | 1000    |
| weighted avg | 0.86      | 0.86   | 0.86     | 1000    |

## ✓ 25. Test the Model on Custom Reviews

The following examples demonstrate how the fine-tuned BERT model can classify new, unseen text.

```
custom_reviews = [
    "This movie was absolutely fantastic. I really enjoyed it.",
    "The movie was boring and a complete waste of time.",
    "The acting was excellent and the story was very interesting.",
    "I hated this movie. It was terrible.",
    "It was an average movie, nothing special."
]

inputs = tokenizer(
    custom_reviews,
    padding=True,
    truncation=True,
    max_length=128,
    return_tensors="pt"
)

inputs = {
    key: value.to(model.device)
    for key, value in inputs.items()
}

model.eval()

with torch.no_grad():
    outputs = model(**inputs)

predictions = torch.argmax(
    outputs.logits,
    dim=1
)

for review, prediction in zip(custom_reviews, predictions):
    sentiment = "Positive" if prediction.item() == 1 else "Negative"

    print("Review:", review)
    print("Prediction:", sentiment)
    print("-" * 80)
```

Review: This movie was absolutely fantastic. I really enjoyed it.  
Prediction: Positive

-----  
Review: The movie was boring and a complete waste of time.  
Prediction: Negative

-----  
Review: The acting was excellent and the story was very interesting.  
Prediction: Positive

-----  
Review: I hated this movie. It was terrible.  
Prediction: Negative

-----  
Review: It was an average movie, nothing special.

Prediction: Negative  
-----

## ✓ 26. Save the Fine-Tuned Model

The fine-tuned BERT model and tokenizer can be saved for future use.

```
save_path = "./bert_sentiment_model"
```

```
trainer.save_model(save_path)
```

```
tokenizer.save_pretrained(save_path)
```

```
print("Model saved at:", save_path)
```

Writing model shards: 100%

1/1 [00:06<00:00, 6.56s/it]

Model saved at: ./bert\_sentiment\_model